

+ Internet Technology (M)

***Design
Specification***

Group DN



Application Overview

Title: Smart Student Task & Deadline Planner

This web application is designed for university students who need to manage multiple courses, assignments, and deadlines efficiently. Many students struggle to keep track of tasks across different modules, which can lead to missed deadlines, poor time management, and unnecessary stress.

The Smart Student Task & Deadline Planner provides a centralised platform where users can create, organise, and monitor their academic tasks in one place. By allowing students to categorise tasks by course, set priorities, and track progress, the system helps them plan their workload more effectively and improve their overall productivity.

This application reduces academic stress, enhances time management skills, and supports students in achieving better academic performance through structured task organisation.

User Stories & MoSCoW Prioritisation

Must (M) – Essential features

1. M1: As a student, I want to register an account so that I can securely access my personal task data.
2. M2: As a student, I want to log in to my account so that I can view and manage my tasks.
3. M3: As a student, I want to create a new task with a title, deadline, and priority so that I can organise my workload.
4. M4: As a student, I want my tasks to be stored in a database so that they are saved and available later.
5. M5: As a student, I want to view all my tasks in a dashboard so that I can see my upcoming deadlines at a glance.

Should (S) – Important but not critical

6. S1: As a student, I want to assign tasks to specific courses so that I can group my work by module.
7. S2: As a student, I want to edit an existing task so that I can update deadlines or details if my plans change.

Could (C) – Nice to have

8. C1: As a student, I want to mark tasks as completed so that I can track my progress.
9. C2: As a student, I want to delete tasks that are no longer relevant.

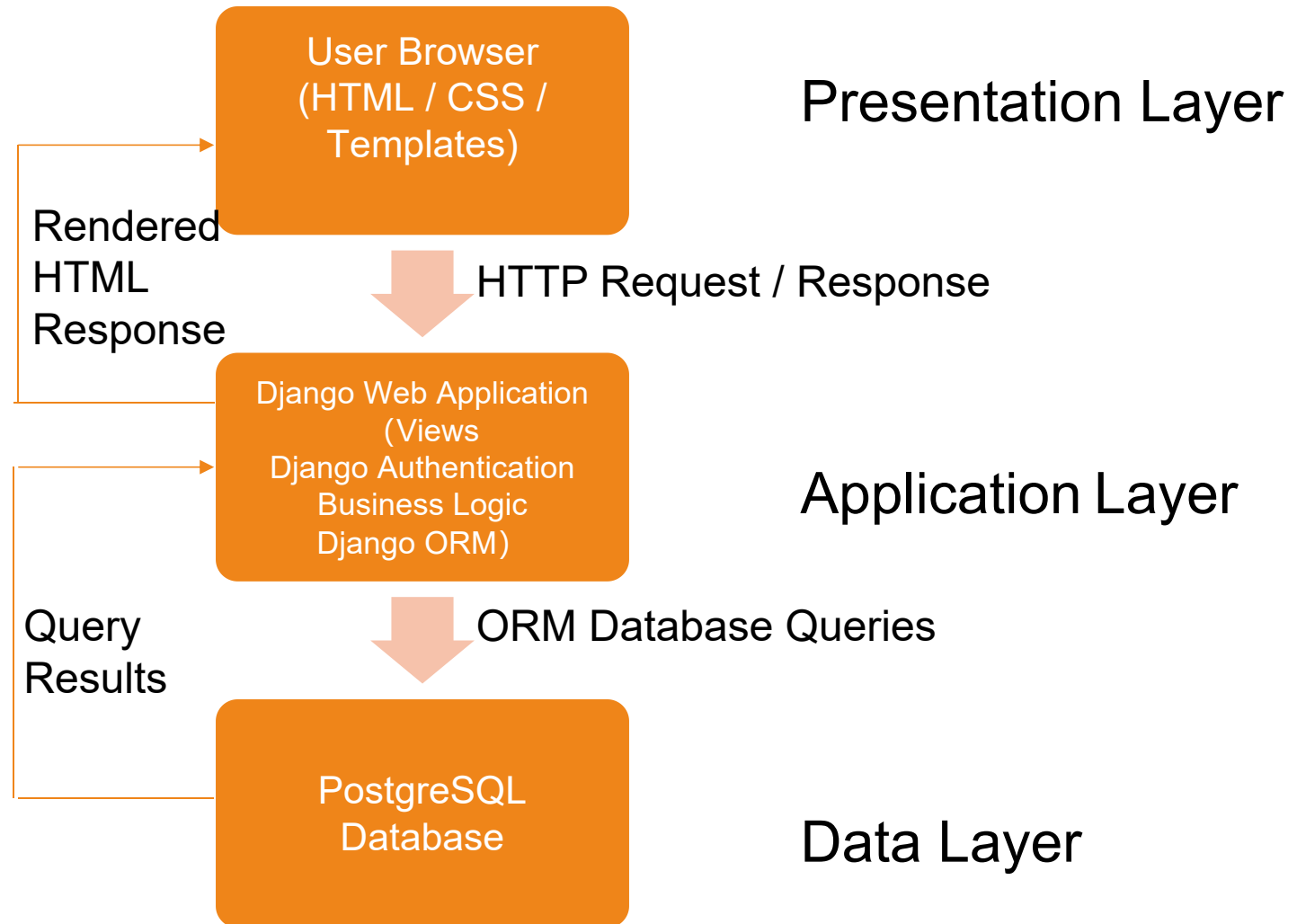
Won't (W) – Out of scope for this version

10. W1: As a student, I want to receive email reminders for deadlines (not implemented in this version).

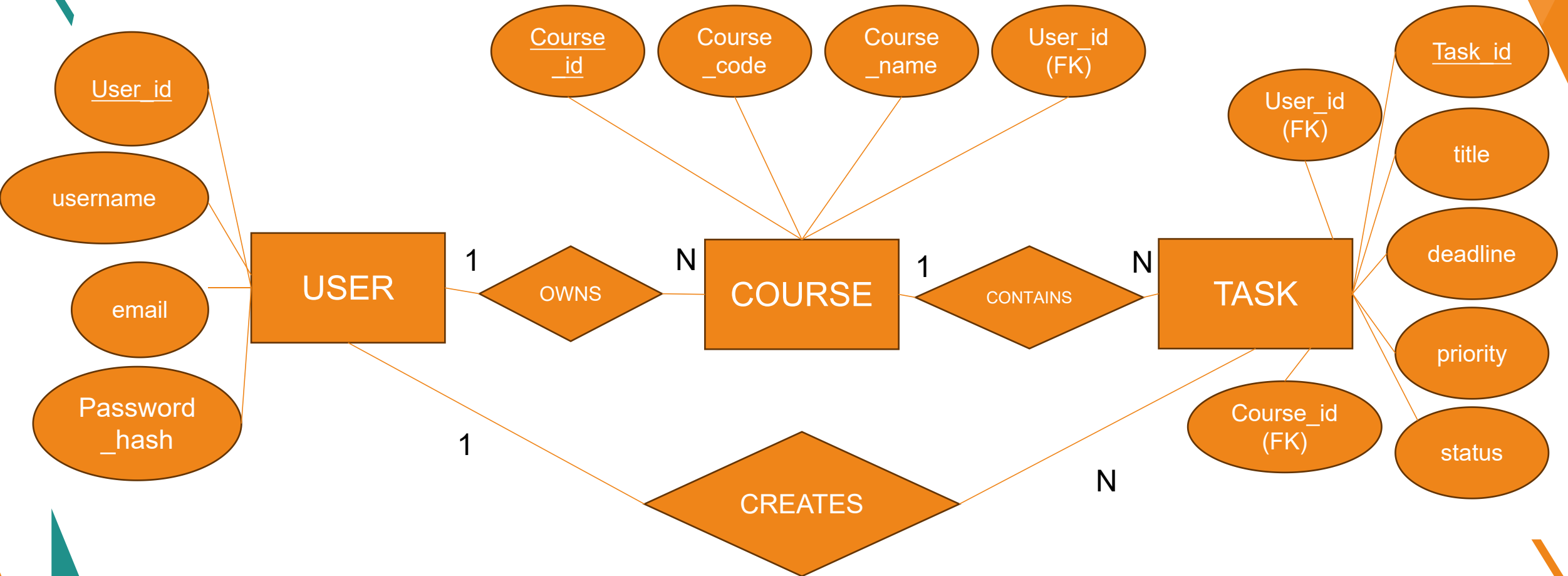
System Architecture Diagram

The system follows a three-tier architecture:

- Presentation Layer: Users interact with the system through a web browser, submitting forms and viewing task information.
- Application Layer: The Django backend processes HTTP requests, handles user authentication, applies business logic, and communicates with the database using the Django ORM.
- Data Layer: A PostgreSQL relational database stores user accounts, courses, and tasks securely.



ER diagram



Entity and Relationship Description

Entity Descriptions

USER

Stores user account information required for authentication and task ownership.

COURSE

Represents academic modules created by users to organise tasks.

TASK

Represents assignments or deadlines created by users. Each task belongs to a specific user and may optionally be linked to a course.

Relationship Constraints

A USER can own multiple COURSES (1:N).

A COURSE can contain multiple TASKS (1:N).

A USER can create multiple TASKS (1:N).

Each TASK must belong to exactly one USER.

A TASK may optionally belong to one COURSE.

Database Design Considerations

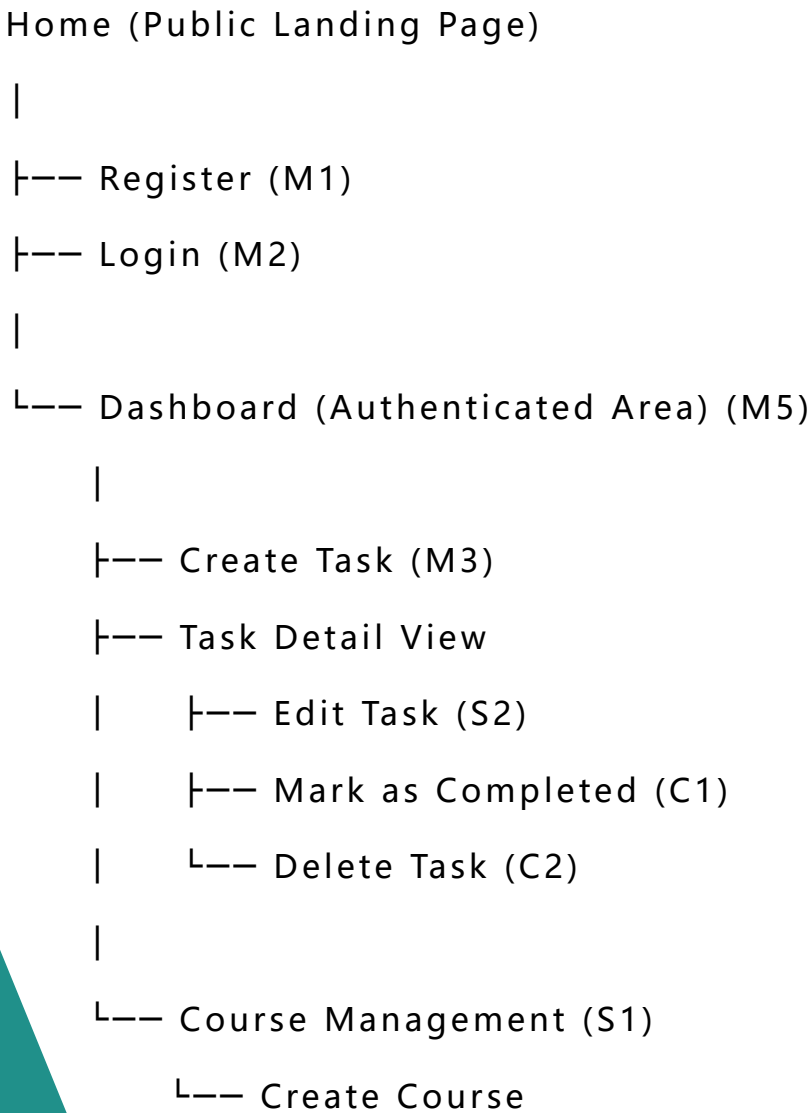
Primary keys uniquely identify each entity.

Foreign keys enforce referential integrity.

Passwords are stored as hashed values for security.

The model ensures data isolation between users.

Site Map & Navigation Structure



Must Requirement Coverage

Must ID	Supported By
M1	Register Page
M2	Login Page
M3	Create Task
M4	Database persistence layer
M5	Dashboard

Wireframe – Secure User Authentication

Key Design Considerations:

Minimal, distraction-free interface

Clear call-to-action buttons

Inline form validation

Accessible labels

Login Page Structure

Email

Password

Login button

Error message display

Design Principle:

Authentication ensures secure isolation of user data and prevents unauthorised access.

Register Page Structure:

Username

Email

Password

Confirm password

Register button

Link to login

Security Considerations:

Password hashing (Django built-in authentication)

CSRF protection

Server-side validation

Wireframe – Task Overview Dashboard

Core Functional Area

The Dashboard acts as the central hub of the application.

Features:

- Display all tasks associated with logged-in user
- Sorted by deadline (ascending)
- Visual priority indicators
- Filter by course (S1)
- “Create Task” shortcut button

Database Interaction (M4)

Tasks retrieved via Django ORM

Query filtered by authenticated user ID

Ensures data isolation

Wireframe – Task Creation Interface

Functional Behaviour:

Form validation (client + server-side)

On submission:

- Task saved to database (M4)
- Linked to current authenticated user
- Redirect to dashboard
- Display success message

Form Fields:

Task Title (required)

Deadline (date picker)

Priority (Low / Medium / High)

Optional Course association

Wireframe – Task Management & State Transitions

Task Detail Page includes:

Edit Task (S2)

Mark as Completed (C1)

Delete Task (C2)

Data State Management

Task Status Values:

Pending

Completed

Design Principle:

The system maintains consistent task state transitions to ensure reliable progress tracking.

Accessibility Strategy

- 1.4.3 Contrast (Minimum)

Text and interactive elements will maintain a minimum contrast ratio of 4.5:1 to ensure readability.

- 2.1.1 Keyboard Accessibility

All functionality (forms, navigation, task actions) will be operable via keyboard fully.

- 3.3.1 Error Identification

Clear, descriptive error messages will be provided for invalid inputs.

- 1.3.1 Info and Relationships

Semantic HTML structure will support screen readers.

- 4.1.3 Status Messages

Dynamic updates (e.g. 'Task saved successfully') will be programmatically determinable.

Appendix

Name	Student ID	Contribution (%)	Responsibilities
Yuzhuo Huang	3075590H	33%	Application overview, user stories, MoSCoW prioritisation
Wenjun Su	3075212S	34%	System architecture diagram, ER diagram, database modelling
Yixiao Wu	3107182W	33%	Site map, wireframes, accessibility strategy

AI Use Statement

We declare that Generative AI was used only to support initial idea brainstorming and minor language refinement.

The system design, ER model, architecture, and overall proposal were independently developed and finalised by the team.